TRABALHO BOOTCAMP-2

NOME: *Fábio Roberto Queiroz Costa Júnior*

RA: *22551751*

SEÇÃO 1: IDENTIFICAÇÃO E ENTENDIMENTO DO PROBLEMA

1.1. Qual é o problema?

O problema central é a dificuldade de organizar e gerenciar tarefas do dia a dia de forma eficiente. Muitas pessoas utilizam métodos manuais (como anotações em papel) ou ferramentas genéricas que não oferecem um fluxo de trabalho claro para criar, acompanhar e finalizar atividades. O **TaskTracker** surge como uma solução digital para centralizar o gerenciamento de tarefas, permitindo ao usuário ter uma visão clara do que precisa ser feito, do que está em andamento e do que já foi concluído.

1.2. Qual a solução proposta?

O TaskTracker será um sistema de linha de comando (CLI) simples e intuitivo. Ele permitirá que o usuário realize as seguintes operações principais:

Adicionar uma nova tarefa com uma descrição/ **Listar** todas as tarefas, com a opção de filtrar por status (ex: "a fazer", "fazendo", "pronto"). /**Atualizar** o status de uma tarefa existente. /**Remover** uma tarefa que não é mais necessária.

1.3. Para quem é esta solução?

Esta solução é ideal para profissionais, estudantes e qualquer pessoa que busca uma maneira simples e organizada de gerenciar suas atividades diárias, sem a complexidade de ferramentas de gerenciamento de projetos mais robustas

SEÇÃO 2: DECOMPOSIÇÃO DE DADOS E REGRAS

2.1. Quais são os dados (entradas e saídas)?

Tipo	Dado	Descrição	Exemplo
ENTRADA	Comando do Usuário	Ação que o usuário deseja executar.	adicionar, listar, atualizar, remover
ENTRADA	Descrição da Tarefa	Texto que descreve a nova tarefa (usado com o comando adicionar).	"Estudar para a prova "
ENTRADA	ID da Tarefa	Número único que identifica uma tarefa (usado com atualizar e remover).	1, 2, 3
ENTRADA	Novo Status	O novo estado da tarefa (usado com o comando atualizar).	"fazendo" ou "pronto"
SAÍDA	Lista de Tarefas	Exibição formatada de todas as tarefas com seus IDs, descrições e status.	[1] Estudar - Status: a fazer
SAÍDA	Mensagens de Confirmação	Texto informando o sucesso ou falha de uma operação.	"Tarefa adicionada com sucesso!"
SAÍDA	Mensagens de Erro	Texto informando ao usuário sobre um erro (ex: comando inválido).	"Comando não reconhecido."

2.2. Quais são as regras de negócio?

1. Cada tarefa deve ter um **ID único e sequencial**. O sistema deve gerar esse ID automaticamente (iniciando em 1) quando uma nova tarefa é adicionada.
2. Uma tarefa, ao ser criada, deve ter o status inicial como **"a fazer"**.
3. Os status possíveis para uma tarefa são: **"a fazer"**, **"fazendo"** e **"pronto"**.
4. A operação de **listar** deve, por padrão, mostrar todas as tarefas. O usuário pode filtrar por um status específico.
5. As operações de **atualizar** e **remover** devem ser feitas usando o **ID** da tarefa. O sistema deve verificar se o ID existe antes de prosseguir.

SEÇÃO 3: RECEITA LÓGICA (ALGORITMO)

algoritmo simples para o sistema.

ALGORITMO TaskTracker

VARIÁVEIS:

tarefas = [] // Uma lista para guardar as tarefas. Cada tarefa será um dicionário {id, descricao, status}

proximo_id = 1

INÍCIO

ENQUANTO VERDADEIRO FAÇA

// 1. Aguarda o comando do usuário

comando = LER_ENTRADA("Digite um comando (adicionar, listar, atualizar, remover, sair): ")

// 2. Processa o comando

ESCOLHA comando

CASO "adicionar":

descricao = LER_ENTRADA("Digite a descrição da tarefa: ")

// Cria uma nova tarefa com status inicial "a fazer"

nova_tarefa = { "id": proximo_id, "descricao": descricao, "status": "a fazer" }

ADICIONAR nova_tarefa à lista tarefas

EXIBIR "Tarefa adicionada com sucesso! ID: " + proximo_id

proximo_id = proximo_id + 1

CASO "listar":

status_filtro = LER_ENTRADA("Filtrar por status? (deixe em branco para listar todas): ")

SE status_filtro == "" ENTÃO

EXIBIR todas as tarefas na lista tarefas

SENÃO

// Cria uma nova lista com apenas as tarefas que têm o status igual ao filtro

tarefas_filtradas = FILTRAR tarefas ONDE status == status_filtro

EXIBIR tarefas_filtradas

FIM SE

CASO "atualizar":

id_para_atualizar = LER_ENTRADA("Digite o ID da tarefa: ")

// Encontra a tarefa com o ID fornecido

tarefa_encontrada = BUSCAR tarefas POR id == id_para_atualizar

SE tarefa_encontrada ENTÃO

novo_status = LER_ENTRADA("Digite o novo status (a fazer, fazendo, pronto): ")

// Verifica se o novo status é válido

SE novo_status for "a fazer" OU "fazendo" OU "pronto" ENTÃO

```
// Atualiza o status da tarefa encontrada
tarefa_encontrada.status = novo_status
EXIBIR "Status da tarefa " + id_para_atualizar + " atualizado com sucesso!"
SENÃO
EXIBIR "Status inválido."
FIM SE
SENÃO
EXIBIR "Tarefa com ID " + id_para_atualizar + " não encontrada."
FIM SE

CASO "remover":
id_para_remover = LER_ENTRADA("Digite o ID da tarefa a ser removida: ")
// Encontra a tarefa na lista e a remove
tarefa_removida = REMOVER tarefas ONDE id == id_para_remover
SE tarefa_removida ENTÃO
EXIBIR "Tarefa removida com sucesso!"
SENÃO
EXIBIR "Tarefa com ID " + id_para_remover + " não encontrada."
FIM SE

CASO "sair":
EXIBIR "Saindo do TaskTracker. Até logo!"
PARAR_LOOP

OUTRO_CASO:
EXIBIR "Comando inválido. Tente novamente."
FIM ESCOLHA

FIM ENQUANTO
FIM
```

SEÇÃO 4: MAPEAMENTO ARQUITETURAL (GIT/GITHUB)

4.1. Estrutura de Pastas do Repositório

Para a Fase 2, o repositório no GitHub será estruturado da seguinte forma:

```
tasktracker/
├── src/                    # Código-fonte principal do projeto
│   └── task_tracker.py     # Arquivo principal com a lógica do sistema
├── README.md               # Documentação do projeto (instalação, uso, etc.)
└── .gitignore              # Arquivos/pastas que o Git deve ignorar (ex: __pycache__)
```

Explicação da Estrutura:

1. tasktracker/: A pasta raiz do projeto.
2. src/: Pasta que contém todo o código-fonte. Manter o código separado da documentação ajuda na organização.
3. task_tracker.py: O arquivo principal em Python que conterá a implementação do algoritmo descrito na Seção 3.
4. README.md: Arquivo essencial que descreve o projeto: como instalá-lo, como executá-lo, quais comandos estão disponíveis, etc.
5. .gitignore: Arquivo que diz ao Git quais arquivos não devem ser versionados (como arquivos temporários ou específicos do sistema operacional).

4.2. Fluxo de Trabalho Inicial com Git

1. Criar um repositório público no GitHub chamado tasktracker.
2. Clonar o repositório para o computador local.
3. Criar o arquivo task_tracker.py dentro da pasta src.
4. Criar o arquivo README.md e o arquivo. gitignore.
5. Adicionar os arquivos ao staged area (git add.).
6. Realizar o primeiro commit (git commit -m "Projeto inicial: estrutura e algoritmo base").
7. Enviar o commit para o repositório remoto (git push origin main).

SEÇÃO 5: LINK DA APRESENTAÇÃO EM VÍDEO

Título: 5. Link do Vídeo

link do vídeo no YouTube: https://youtu.be/Ytm-h373A3o